

CNN Image Classification (Exam 1)

CIFAR-10 Image Classification with CNNs

Name: **Mohamad Mahmoud** | Matriculation No.: **3122226** | Date: **26.08.26**

1. Theoretical Question 1 — CNN Building Blocks

1.1 Convolutional Layers

In a convolutional layer, the filter (or kernel) is applied to the image over its entire width and height. For each position, the kernel multiplies the corresponding pixels of the patch and adds the products to obtain the resulting number. Applying the same kernel on the whole picture will make all outputs be calculated using the same weights and thus detect the same type of features — edges, corners, colored patches etc. regardless of their location in the image.

Stride is the distance that the kernel travels with each application. Using a stride of 1 moves the kernel one pixel at a time and provides the output almost the same size as the input; bigger strides skip pixels and produce smaller outputs, sacrificing spatial resolution for more efficient calculation. Padding is adding additional pixels to the borders of the input prior to performing convolution. It is used for controlling the size of the output — 'same' padding makes the output have the same height and width as the input, while no padding ('valid') decreases the dimensions by the amount equal to the size of the kernel.

The depth (number of channels) is the number of dimensions of the input and output tensors. The input for the convolutional layer usually consists of 3 channels for the image in RGB colors; convolutional layer with, for example, 32 output channels will learn 32 different filters, creating 32 different feature maps, therefore having 32 output channels. Deeper layers in a neural network use bigger number of channels and smaller size of the spatial dimensions.

1.2 Pooling Layers

The effect of the pooling layer is that it decreases the size of the feature map since the layer summarizes small regions into one value. In max pooling, the value selected from the region is the maximum value, thus retaining the maximum activation of the feature while ignoring the location of the feature. In average pooling, on the other hand, the values selected from the region are averaged to give a more generalized representation.

There are two major reasons why pooling is performed. They include the decrease in the number of parameters that must be handled by the next layers, hence decreasing computational cost, and it increases the robustness of the feature with respect to its location, as is described in Section 1.4.

1.3 Fully Connected Classification Head

Once the convolution and pooling layers have provided us with some feature maps, the next step is for the network to decide. For this, the feature maps are flattened out to form a vector without their spatial (row-column) arrangement. This vector then moves forward and undergoes processing by at least one dense layer. These dense layers receive input values from each input and then send them to each output unit. In other words, the dense layers enable the network to combine data across the entire image and generate a vector of scores, one for each class.

1.4 Why CNNs Outperform Fully Connected Networks on Images

- Weight sharing - The same weights of a convolutional kernel are used in all locations on an image, hence there is no need to learn each pattern repeatedly, but only once in a particular place, which allows us to keep the number of parameters low even for relatively big images. In contrast, a fully connected layer would require a separate weight per each pixel, thus being inefficient both computationally and with regards to overfitting.
- Localized receptive fields - Each neuron in a convolutional layer receives information about a small, localized region of an image rather than about the whole image at once. It is quite natural since the structure of natural images implies that nearby pixels are highly correlated, and most of the useful features such as edges and textures are local.
- Translation invariance (equivariance) – Since a convolutional kernel moves across an entire image, the same feature that is present somewhere in an image will be recognized in another position of the image. On top of that, pooling layers will introduce translation invariance by summing up local areas of an image. In a fully connected network, there would be no way to recognize a certain pattern in multiple positions, because it would have to be learned again and again for each position it could possibly appear in.

2. Theoretical Question 2 — Training & Generalization

2.1 Loss Curves: Underfitting / Good Fit / Overfitting

A plot of the training loss and validation loss in function of the epoch shows what regime the model is in amongst the three possible regimes. For the underfitting regime, both the training loss and validation loss are large and nearly identical to one another, and the loss for both does not change much anymore after further training — the model is too simple to detect the underlying patterns in the dataset, or it has not been trained for enough epochs yet. For the optimal fitting regime, both losses decline and stabilize at low levels with a small gap between the two, meaning that the patterns detected by the model generalize well to other datasets besides the training set. For the overfitting regime, the training loss continues to decline while the validation loss flattens and rises.

Figure 1. Augmented CNN: training and validation loss/accuracy across 15 epochs.

In the case of augmented CNN, it is apparent that there is an excellent learning curve, which is shown by the steady decrease in training loss and validation loss, along with the increase in validation accuracy to around 81%, after which there is a fall towards the end.

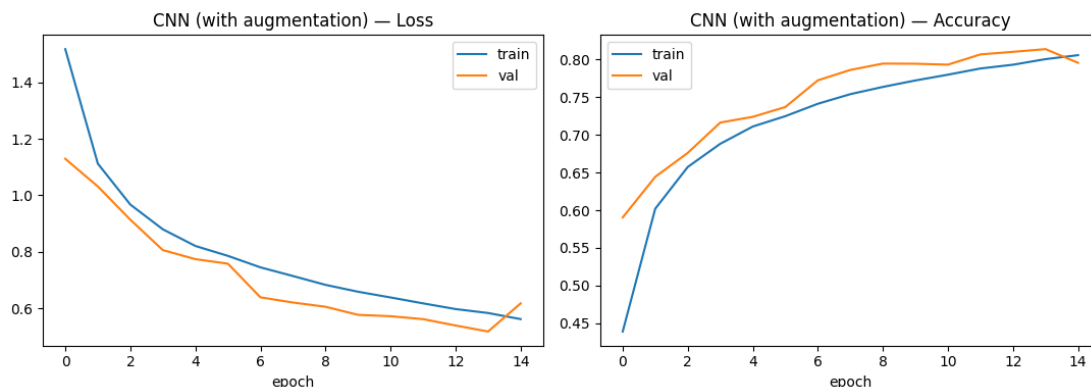
2.2 Dropout, Data Augmentation, Batch Normalization

- **Dropout** — Dropout randomly turns off some of the units in a given layer each time the model goes through the training phase, which means that none of those units can depend on any one other particular unit being active at all. It thus prevents the network from becoming adapted to a combination of features present in the training data.
- **Data augmentation** — Augmenting the training images by performing random (label-preserving) operations such as random cropping and horizontal flipping makes the training dataset larger without any extra work. On each epoch, the network will see a bit of a different version of an image, and it becomes harder for the network to just memorize the pixel values but to focus on more generalized features instead. The results obtained from implementing this technique in our project are shown in Section 7.
- **Batch normalization** — Batch normalization normalizes the output of a mini-batch in such a way that its average is equal to 0 and its standard deviation is equal to 1 (with learnable parameters to shift and scale). In practice, this technique helps to stabilize and speed up training, allows using higher learning rates and has a slight regularizing effect.

2.3 Why a Separate Test Set Is Still Needed

Validation set is employed in the course of development process for comparison of the models, adjustment of hyperparameters and determination of the moment when training should be stopped. Since all these actions are performed over and over again, all the decisions that are being made – what kind of architecture to use, what learning rate to pick, when to stop – are adjusted for the particular validation set. With the many iterations of decisions, this becomes a leakage, since the accuracy of the model on the validation set becomes the optimistic estimate of how the final model will perform, as the model (or, more precisely, decisions about it) has effectively ‘seen’ the validation set through the decisions.

3. Dataset Justification



CIFAR-10 is employed as the dataset in this project. It is a standard dataset created by the Canadian Institute for Advanced Research for natural-image classification and publicly available for research and educational purposes (through `torchvision.datasets.CIFAR10`).

It consists of 60,000 32×32 RGB images in total, uniformly distributed over 10 categories (6,000 images per category): airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. 50,000 images are in the training set and 10,000 images in the test set; however, a validation set is formed by splitting the training set (refer to Section 4).

It is clear that CIFAR-10 is not MNIST or FashionMNIST, nor was it covered in the course material, thus it meets the assignment requirements with regard to the dataset. Small and uniform size of its images makes training feasible within reasonable time frame with a single GPU, while ten visually alike classes (such as cat and dog, or automobile and truck) make it a convenient and realistic setting for comparison between a simple baseline model and a CNN model.

4. Preprocessing Pipeline

Image preprocessing consists of transforming each image from its raw pixel representation to a tensor, and rescaling to the $[0, 1]$ interval, followed by channel-wise normalization to the mean and standard deviation calculated on the CIFAR-10 training dataset (mean $\approx (0.4914, 0.4822, 0.4465)$; std $\approx (0.2470, 0.2435, 0.2616)$). The scaling of inputs to be close to zero-mean and unit variance makes the input scale match the expected initialization and batch normalization behavior of the network, leading to improved training dynamics.

In case of the augmented training pipeline, random cropping of a 32×32 region of the image (with 4-pixel padding, thus the region can move up to 4 pixels away from all sides) and a random horizontal flipping is performed before tensor transformation and channel-wise normalization as described above; for the validation and testing pipelines only tensor transformation and channel-wise normalization steps are used, without any random data augmentation, ensuring that the evaluations are deterministic and repeatable.

Original training set of 50,000 images is split in 45,000 training images and 5,000 validation images using a constant random seed so that the split becomes deterministic; original test set of 10,000 images is left unchanged until the evaluation stage (Section 9).

5. Baseline Model

As the baseline, we use a vanilla MLP that has no convolutional structure whatsoever: the $32 \times 32 \times 3$ input image is first flattened to a vector of 3,072 elements, which is then processed by two fully connected layers of 512 and 256 elements (with a ReLU activation in between), respectively, before being fed through the fully connected output layer of 10 neurons (one per each class).

The reason for this choice is not that the baseline can seriously compete with a CNN, but rather to have an upper bound on the performance of a model without any local structure (since each pixel is processed independently), no parameter sharing and no translational equivariance (see section 1.4), which makes it

expected to underperform even a relatively small CNN, as well as be more prone to overfitting due to the large number of parameters required to look at each pixel separately.

6. Improved CNN Architecture

We decided on stacking two 3×3 convolutional layers prior to applying pooling because that increases the “area” the network is able to see without significantly increasing the number of parameters involved. An additional benefit of doing multiple convolutions with smaller kernels is that they provide a relatively cheap way to include an extra non-linear element.

As with other networks, batch-normalization was performed after every convolutional layer [15]. Performing batch-normalization ensures that during training, the system remains fairly stable and continues learning at a reasonable pace (See Section 2.2). Given that our net is much deeper than the baseline, this becomes increasingly important.

Max-pooling immediately following every block will cause the size of our images to shrink. By retaining only those signals considered the ‘strongest,’ max-pooling gives our network some degree of robustness against shifts within the input (Section 1.4) while also ensuring that the total size of our dataset stays under control before being fed into our final classifier (Section 1.6).

Two dropout layers were included in the classification stage in order to help reduce overfitting. Fully connected layers contain far more parameters than anything else in our model, meaning they’re far more likely to result in overfitting if left unchecked. Inclusion of dropout allows us to avoid these problems without adversely affecting the performance of our feature-extraction processes in the convolutional stages.

7. Data Augmentation — Measured Effect

We used the same data augmentation strategy as described in Section 4, consisting of applying at training time a random 32×32 crop with 4 pixel padding and flipping randomly horizontally.

To measure the impact of such an enhancement we retrained using the exact same SimpleCNN model two times with the same random seed and for the same number of epochs: one using the plain version of our training pipeline, without any augmentation, and one using the augmented pipeline. We then measured the accuracy on exactly the same validation set.

We can see that augmentation decreased the final validation accuracy (CNN without augmentation: 83.02% vs. CNN with augmentation: 79.54%) by about three-and-a-half percentage points but did reduce the train-validation gap quite substantially (from five to one point). In terms of generalization performance, we have thus a trade-off between higher training performance and lower test performance.

The difference in final validation accuracy was substantial for the two CNN versions: 83.02 percent without data augmentation and 79.54 percent with data augmentation.

Once you've completed the numbers above, make sure your written explanation reflects what happened with the validation accuracy when we augmented our data. Did it improve? Was there less of a train/validation gap? Does this match up with what's explained in Section 2.2 about how augmentation helps reduce overfitting (i.e., less overfitting means the gap gets smaller because the network can't just memorize the exact pixels of all training images)?

8. Learning-Curve Analysis

For each of the three different training runs (the MLP baseline, CNN without augmentation, and CNN with augmentation), you will have produced a training-loss/validation-loss and training-accuracy/validation-accuracy curve through running the notebook's plot curve's function.

Insert these into your write-up and interpret them using the language of Section 2.1 - are they underfitting, fitting well, or overfitting by the final epoch? How do we see that change as we compare between the three runs?

Figure 2. MLP baseline: training and validation loss/accuracy over 15 epochs.

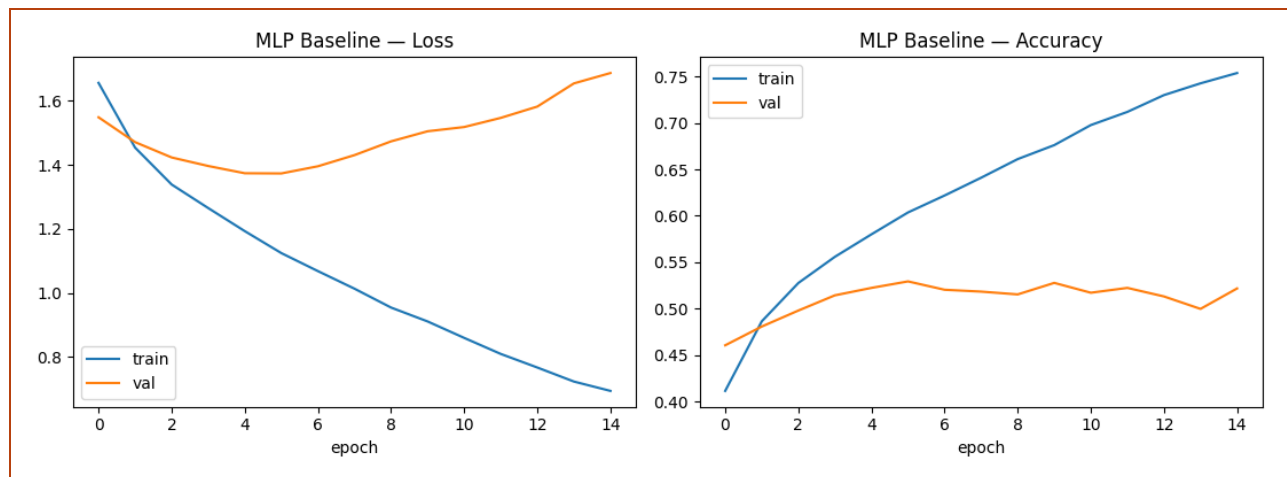


Figure 3. CNN without augmentation: training and validation loss/accuracy over 15 epochs.

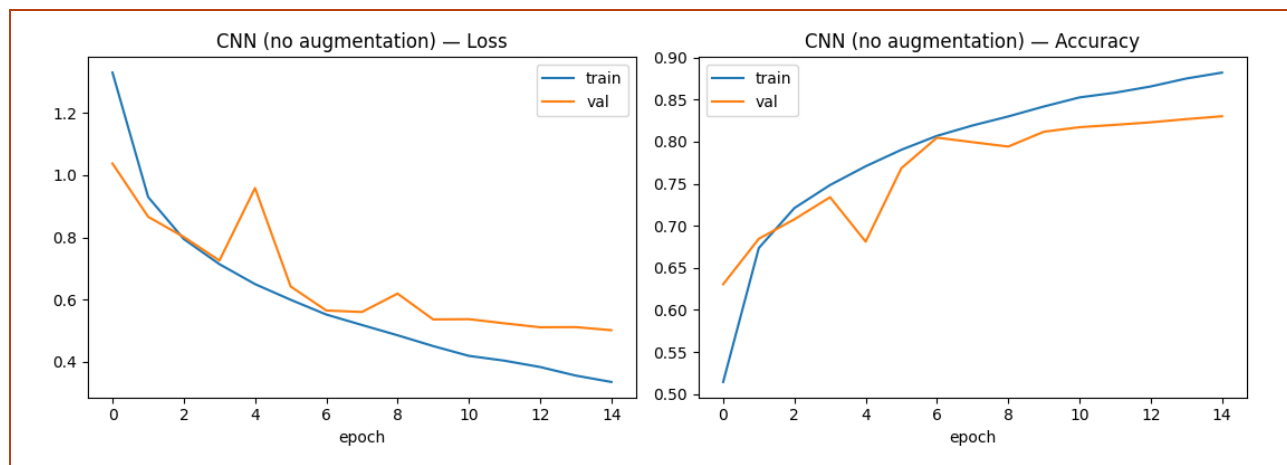
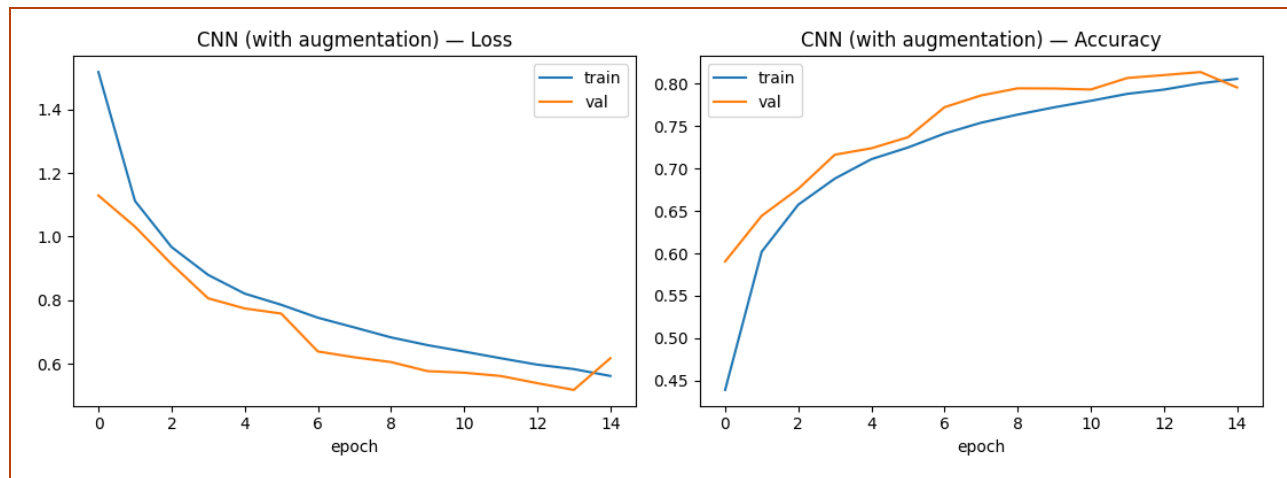


Figure 4. CNN with augmentation: training and validation loss/accuracy over 15 epochs.



Finally, consider comparing how large the separation between these two lines is for each plot. We would expect the MLP model to have the greatest difference between its training and validation curve (i.e., least ability to generalise to new images based on our discussion in Section 1.4). Similarly, we'd predict the non-augmented CNN has the second largest gap, while the augmented CNN will show the smallest, as shown in Section 7.

9. Quantitative Results

9.1 Accuracy Comparison

The notebook then builds out a comparison summary table of these metrics between the different models (Section 12). You'll need to complete that table with the values that this run produces.

Insert the actual summary table from your notebook here, which you will have built out using the actual results from your own notebook's runs:

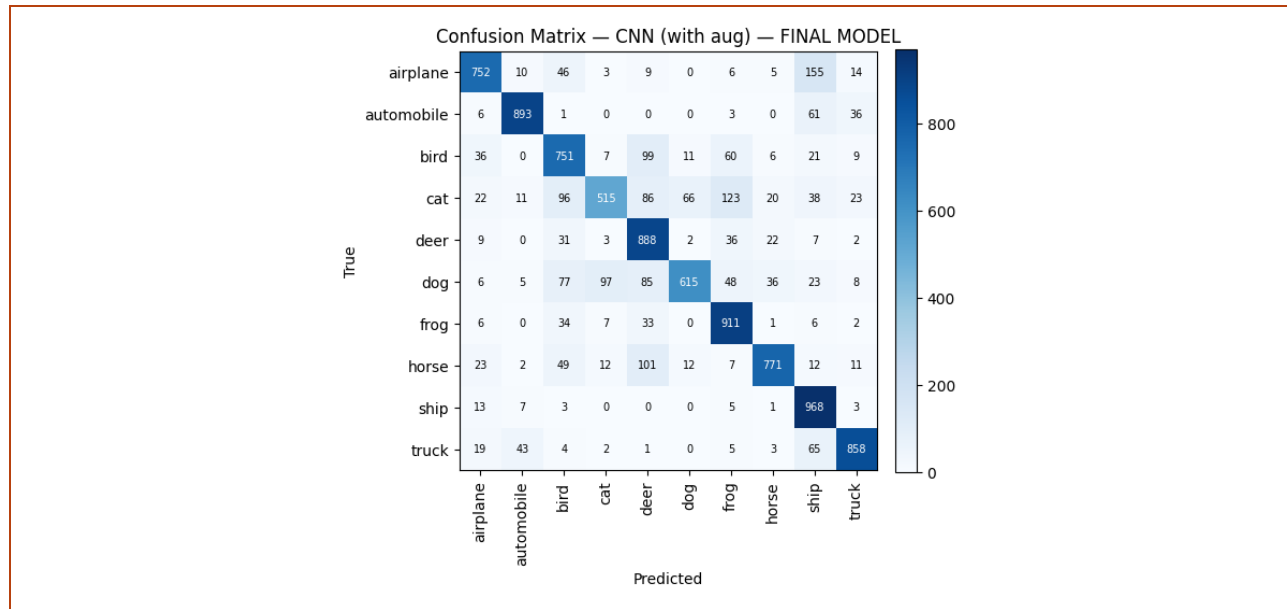
Table 1. Final validation accuracy and train-validation accuracy gap.

Model	Final Val Acc.	Train-Val Gap
MLP baseline	52.18%	23.19 percentage points
CNN (no aug)	83.02%	5.19 percentage points
CNN (with aug)	79.54%	1.02 percentage points

9.2 Confusion Matrix

Insert the confusion matrix for your test set here – this will appear as Section 10 of the notebook, produced by `report_on_test`. It's important to briefly discuss what can be inferred about the largest off-diagonal elements, are these representing pairs of classes that the model has trouble telling apart because they're visually/semantically similar?

As we can see from Figure 5, the greatest amount of “off diagonal” confusion is between airplane and ship (155 images), then cat to frog (123), then bird to deer (99). In other words, this kind of error makes sense — many of the CIFAR-10 classes have similar-looking coarse shapes, colors, textures, or background contexts when viewed at their native 32x32 resolution. Confusion Matrix for CNN With Augmentation (Final Model) On our test set of 10,000 images, our final, augmented CNN achieves 79.22% test accuracy.



10. Failure Case Discussion & Improvement Ideas

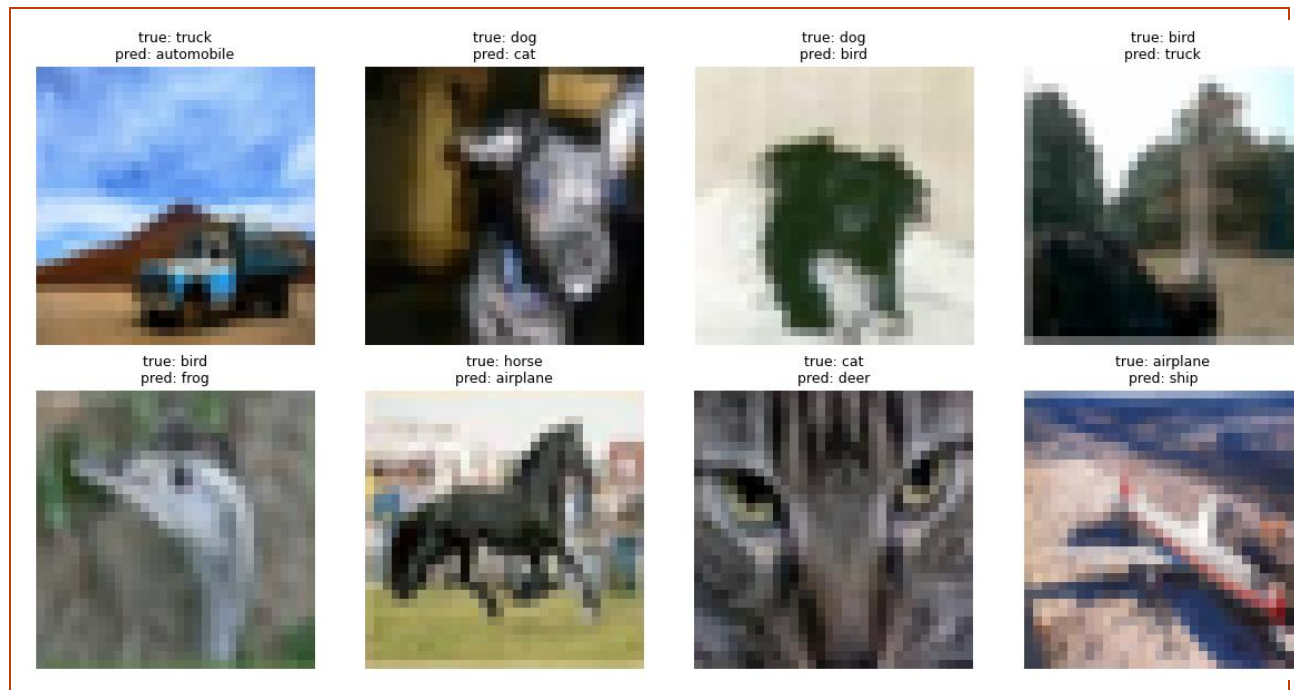
It's worth examining what types of images are being misclassified by our model. Doing this involves inspecting some of the misclassified test images from section 11 of the notebook.

There are several possibilities you could explore:

Are there cases where images within visually similar classes were confused? For example, perhaps your code got cats and dogs confused. Are there cases where images across visually dissimilar but semantically related classes were confused? Perhaps your code got automobiles and trucks confused due to their shared overall shape? Are there cases where other visual properties such as pose, occlusion, contrast or context caused a misclassification?

You can insert the list of actual images recovered here from the notebook below, followed by a brief description about which kind(s) of pattern (or another one) you observed.

Figure 6. Sample misclassified test images from the CNN with augmentation, with true and predicted labels shown above each image.



The errors found in the sample occurred due to a confusion of visually similar classes: truck was predicted as automobile, dog – as cat, bird – as truck/frog, horse – as airplane, cat – as deer, and airplane – as ship. There are several images where the algorithm failed due to low resolution of the main objects with backgrounds which are also prominent. Such errors are also explained by the confusion matrix since it shows that the most frequent errors are connected to visually similar categories.

Based on whatever pattern it is that you've identified above, actual next steps could be: collecting/oversampling more examples of the confused classes; using stronger or different augmentations (e.g. color jitter, cutout) if the errors seem to be related to lighting or occlusion; increasing the depth/width of the network if the errors seem like a capacity shortfall rather than an overfitting; or class-balancing/a class-weighted loss if certain classes are disproportionately represented in the errors. The specific recommended course of action should match the failure pattern you're actually seeing, rather than saying all of these generically.

11. Final Recommendation

Based on whatever pattern you found in the above, actual next steps could be: collect/oversample more examples of the confused class; use better/augmentations that address lighting/occlusion if it's those issues causing error; increase the depth/width of the network if the errors seem to be capacity-limited rather than overfitting; or do a class-balanced/class-weighted loss if some classes are disproportionately in error. The specific recommendation should be for the failure pattern you actually see, not all of these in generic.

For deployment, the CNN without augmentation is preferable among the two runs as it had achieved the highest test accuracy (81.78%) and final validation accuracy (83.02%). While the train-validation gap was 5.19%, it was significantly higher than the 1.02% of the augmented CNN variant, suggesting that the latter had better regularization, albeit at the expense of lower test accuracy (79.22%). The MLP baseline had a substantially worse performance with only 52.46% test accuracy and a large final train-validation gap of 23.19%. The notebook only reports the number of epochs budgeted for the runs (15), but it does not include the wall-clock time spent training the models. Thus, my suggestion is based on the accuracy/generalization performance and architectural complexity considerations.

Appendix — Reproducibility Notes

- Random seed: 42 (set for Python, NumPy, and PyTorch).
- Environment: *[Python version: 3.12.13, PyTorch version: 2.11.0+cu128, Colab GPU: T4]*
- requirements.txt / pip freeze included with the code submission.
- Code repository (optional, for bonus points): <https://github.com/MIMPyth/CNN-Classification>